SDG BLOCKCHAIN ACCELERATOR

Prototype (PoC) Report

In collaboration with
UNDP Bosnia and Herzegovina / University of Sarajevo) / Sala

1. Project Information

- **Project Name:** Digital Identity & Credential Verification
- **Challenge & UNDP Office:** UNDP Bosnia and Herzegovina & Challenge Owner: University of Sarajevo
- **Document Version:** V 1.0 January 2026

2. Project Overview

*This Proof of Concept (PoC) implements a **decentralized certificate verification system** using the Cardano blockchain.*

The system enables educational institutions to issue digitally verifiable academic credentials, while allowing third parties to verify authenticity via QR code and blockchain-anchored proofs.

Key features include:

- Admin portal for certificate issuance and management
- Student-facing verification interface
- QR-based public verification flow
- Hash anchoring of credential data on Cardano
- Off-chain storage of certificate documents (PDF/images)
- Sandbox environment for live testing and demonstration

The PoC focuses on **credential integrity, tamper resistance, and interoperability**, while remaining compliant with privacy-by-design principles.

3. Repository Structure

(Outline how your code and related files are organized.)

Suggested Structure:

```
/prototype
    /contracts   → Aiken smart contract source code
    /tests       → Unit and integration tests
    /scripts     → Deployment scripts and CLI commands
    /docs        → README.md, deployment guide, screenshots
```

4. Build Instructions

(Explain how to compile/build the smart contracts.)

Example:

```
# Compile Aiken contracts
aiken build
```

- Include any environment variables or dependencies required.
- Mention compatible Aiken CLI and Cardano-node versions.

5. Test Instructions

(Explain how to run unit and integration tests.)

Example:

```
# Run unit tests
aiken test

# Run integration/emulator tests
aiken emu
```

- Include expected outputs or example logs.
- Mention edge cases tested.

6. Deployment Instructions

(Explain how to deploy contracts to testnet or mainnet.)

Example:

```
# Build transaction
cardano-cli transaction build \
  --testnet-magic 2 \
  --tx-in <UTXO> \
  --tx-out <ADDRESS>+1000000 \
  --change-address <CHANGE_ADDR> \
  --out-file tx.raw
```

```
# Sign transaction
cardano-cli transaction sign \
  --tx-body-file tx.raw \
  --signing-key-file payment.skey \
  --testnet-magic 2 \
  --out-file tx.signed

# Submit transaction
cardano-cli transaction submit \
  --testnet-magic 2 \
  --tx-file tx.signed
```

- Include steps for deploying multiple validators if relevant.
- Mention any prerequisites (e.g., testnet ADA, cardano-node running).

7. Testnet / Emulator Results

(Provide evidence of successful deployment and contract execution.)

Example Table:

Transaction ID	Contract Action	Status	CPU	Memory	Notes
abc123	Mint Token	Success	310k	900k	Validator executed correctly
def456	Transfer Token	Success	280k	850k	Edge case tested: invalid redeemer rejected

- Include screenshots of dashboard or transaction confirmations if available.

8. Dependencies & Environment

- Aiken CLI version
- Cardano-node version
- Testnet used (Preview, Preprod)

- Any libraries, SDKs, or off-chain services

9. Demo / Walkthrough

A live sandbox environment is available:

🔗 **Sandbox URL:**
https://certificate-verification.dev.sandbox.sala.tech/

The demo demonstrates:

- Certificate verification via QR code
- Credential lookup and validation
- Blockchain anchor confirmation
- User-facing verification UI

Screenshots and demo materials are included in the documentation folder.

10. Remaining Issues / Next Steps

- *Finalize digital identity integration (W3C Verifiable Credentials alignment)*
- *Complete API-to-smart-contract connection*
- *Extend test coverage for edge cases*
- *Migrate from Preview to Preprod testnet*
- *Finalize IPFS document storage integration*
- *Conduct performance and load testing*